Latency-Constrained Request Routing for Interactive LLM Applications: A Multi-Armed Bandit Approach

Min-su Jeon[a], Minkyung Kim[a], Sohee Bae[a], Sangdon Park[b,*]

[a] *Sayberry Games Inc., Daejeon, Republic of Korea*
[b] *Department of Computer Science, School of Software Convergence, Daejeon University, Daejeon, Republic of Korea*

Abstract

This paper targets *sub-second* interactive LLM applications—conversational agents, narrative games, short-form customer support—where per-request output is bounded to a few hundred tokens and the latency budget is $L_{\max} \approx 500$ ms. In this regime, cloud-based LLMs offer superior quality but incur network delays, while edge-deployed models provide low latency but limited capability. This paper formulates request routing as an online learning problem and proposes EdgeLLM-Router, a Multi-Armed Bandit (MAB) framework that adaptively routes queries to edge, cloud, or hybrid execution paths.

The paper makes four contributions: (1) a lightweight query complexity predictor (a 1,187-parameter MLP) that reaches 80% three-way accuracy and degrades gracefully under feature perturbation; (2) a multi-armed bandit formulation with a latency-aware reward and a tail-sensitive Deadline-Aware Thompson Sampling (DATS) algorithm whose arm score incorporates a conservative upper-tail latency estimate and a recency-weighted deadline-miss signal; (3) an open-source edge computing simulator with queueing, batching, cold-start, and heavy-tail network effects, released with configuration files and the MLP weights;[1] and (4) comprehensive experiments across three application domains (narrative, customer-QA, code) that include cascade and QoS-driven baselines.

On an interactive narrative workload, EdgeLLM-Router reaches 99.2% of cloud-only quality while reducing p_{99} latency by 64% and SLA violations by 99% compared to the cloud-only baseline, with 87% cost savings. A paired bootstrap test over the 5-seed main workload shows DATS significantly reduces SLA violations compared with plain Thompson Sampling (95% CI excludes zero), and a separate network-shift experiment shows the same ordering under non-stationary conditions. Analysis across nine experimental dimensions—including latency-model sensitivity, multi-domain generalisation, predictor ablation, and statistical robustness—validates the effectiveness of adaptive routing.

Keywords: Edge computing, Large language models, Multi-armed bandits, Request routing, Latency optimization

1. Introduction

Large Language Models have transformed interactive applications, from conversational agents to narrative games. These applications demand both high response quality—coherent, contextually appropriate outputs—and low latency to maintain user engagement. Research on response time thresholds indicates that delays beyond 100 ms–1000 ms range increasingly disrupt the perception of direct interaction (Nielsen, 1993); for conversational applications, a practical threshold of 500 ms is commonly adopted.

Cloud-hosted LLMs (GPT-4, Claude, Gemini) deliver state-of-the-art quality but introduce 100–2000 ms of network latency depending on request complexity and network conditions. Edge-deployed models (quantized LLaMA, Phi, Gemma, or cloud-edge services like Gemini Flash) run with 10–100 ms latency. While modern edge models have narrowed the quality gap with cloud, they may still exhibit

*Corresponding author
Email addresses: jms535@sayberrygames.com (Min-su Jeon), mkkim1778@gmail.com (Minkyung Kim), bread@sayberrygames.com (Sohee Bae), sangdonpark@dju.kr (Sangdon Park)

higher latency variance and SLA violations under load.

Not all queries require the same capability. Simple acknowledgments ("I understand"), short factual lookups, and templated responses can be handled by edge models. Complex reasoning, multi-turn context integration, and creative generation benefit from cloud models. An intelligent router that predicts query complexity and selects the appropriate execution tier can achieve both quality and latency.

However, query complexity is not directly observable—it must be inferred from input features. Network conditions fluctuate unpredictably. User preferences and application requirements vary. These uncertainties call for an *online learning* approach that adapts routing decisions based on observed outcomes.

Scope. This work explicitly targets *sub-second interactive* workloads: conversational NPC dialogue, customer-support turns, and short narrative generations, where per-request output is bounded to a few hundred tokens and the end-to-end deadline is typically $L_{\max} \in [300, 1000]$ ms. We use $L_{\max} = 500$ ms throughout as the canonical deadline. Longer-form generation—multi-paragraph document QA, full source-file synthesis—falls outside this regime because the per-request decoding time alone can exceed the deadline regardless of which tier is chosen. Section 6.6 reports code-generation results as a boundary case that motivates either a larger $L_{\max}$ or streaming partial outputs; that dimension is orthogonal to the routing problem considered here.

This paper makes the following contributions:

- **Problem formulation.** Request routing is formalized as a multi-armed bandit problem with latency-aware rewards. Each arm corresponds to an execution tier (edge, cloud, hybrid); a lightweight complexity predictor provides routing guidance; rewards combine quality with latency penalties.

- **Query complexity predictor.** A lightweight classifier (< 1 ms inference) estimates query complexity from token count, entity density, question type, and context length. The predictor is a two-layer MLP (hidden size 16, 1,187 parameters) trained by softmax cross-entropy on a 6,000-example synthetic dataset spanning three application domains with 5% label-noise to emulate human disagreement; it reaches 80% three-way accuracy on a 1,500-sample held-out set and degrades by 18 pp under σ=0.2 Gaussian feature perturbation (Section 4.1).

- **MAB routing algorithms.** This paper implements and compares UCB1, Thompson Sampling, and ϵ-greedy variants adapted for latency-aware settings. The proposed *Deadline-Aware Thompson Sampling* (DATS) differs from plain Thompson Sampling in three ways: (i) an upper-tail latency estimate $\hat{\ell}_k + \kappa\hat{\sigma}_k$ is used inside the deadline-probability term rather than the mean alone, so that heavy-tail spikes reduce the arm score; (ii) a recency-weighted miss-rate signal directly penalises arms that have recently violated deadlines; (iii) the deadline penalty is scaled up adaptively when any tier is showing sustained misses, which accelerates reaction to regime changes. These modifications make DATS measurably more robust than Thompson Sampling under non-stationary conditions while preserving comparable performance in stationary workloads.

- **Edge computing simulator.** An open-source simulator is released that models heterogeneous edge-cloud topologies, realistic network traces, and LLM inference characteristics. The revised simulator now models queueing delay (M/G/1-style utilisation), continuous batching on the cloud tier, probabilistic cold-start penalties, explicit time-to-first-token and per-token decoding, and a Pareto-mixed heavy-tail for cloud variability. Configuration files, MLP weights, and experiment scripts are released at `https://github.com/Sangdon-Park/edgellm-router`.

- **Comprehensive empirical evaluation.** On traces from an interactive narrative application, EdgeLLM-Router achieves 99.2% of cloud-only quality while reducing SLA violations by 99% and API costs by 87%. Nine experimental analyses are presented covering quality-SLA trade-offs, tier selection, non-stationary adaptation, cost efficiency, statistical robustness (paired bootstrap), latency-model sensitivity, multi-domain generalisation, predictor ablation, and a new baseline compar-

ison against cascade (FrugalGPT-style), QoS-static, and cost-aware scheduling policies.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the routing problem. Section 4 presents the proposed algorithms. Section 5 describes the edge computing simulator. Section 6 reports experimental results. Section 7 discusses implications and limitations. Section 8 concludes.

2. Related Work

This work intersects edge LLM inference, adaptive model serving, and online learning for routing.

Edge LLM inference. Quantization (GPTQ (Frantar et al., 2023), AWQ (Lin et al., 2024)) and distillation ((Hinton et al., 2015; Sanh et al., 2019; Ho et al., 2023; Hsieh et al., 2023)) reduce model footprint, while runtimes like MLC-LLM (Team, 2023) and llama.cpp enable deployment on consumer hardware. These techniques preserve $> 90\%$ of original quality on simple tasks but degrade on complex reasoning; modern edge-optimised models (Gemini 3.1 Flash Lite (Google DeepMind, 2026a)) have narrowed the gap but not closed it. The observed complexity-dependent gap motivates adaptive routing instead of uniform edge deployment.

Adaptive model serving. Clipper (Crankshaw et al., 2017), InferLine (Crankshaw et al., 2020), and SHEPHERD (Zhang et al., 2023) route requests across heterogeneous backends using batching and learned cost models. LLM-specific systems include Orca's continuous batching (Yu et al., 2022), vLLM's PagedAttention (Kwon et al., 2023), and FlexGen (Sheng et al., 2023). These optimise *single-model* serving and are complementary to the *multi-tier routing* problem addressed here. Cascade approaches (FrugalGPT (Chen et al., 2023b), RouteLLM (Ong et al., 2025), mixture-of-thought (Yue et al., 2023), AutoMix (Aggarwal et al., 2024)) route via fixed confidence thresholds or supervised classifiers; we formulate routing as online learning that adapts without ground-truth labels, and include FrugalGPT as a baseline (Section 6).

Multi-armed bandits and constrained routing. UCB1 (Auer et al., 2002) and Thompson Sampling (Thompson, 1933; Russo et al., 2018) achieve $O(\sqrt{KT \log T})$ regret for stochastic bandits; contextual variants (Li et al., 2010; Riquelme et al., 2018) use side features. Constraint-aware formulations include conservative bandits (Wu et al., 2016) and knapsack bandits (Badanidiyuru et al., 2018); we adopt a practical penalty-based approach instead. A closely related body of work uses MAB formulations to optimise routing and resource allocation in wireless and multi-hop networks, where the feedback structure—selecting one of several channels or paths whose quality is observed only after use—mirrors the edge-cloud LLM setting. Zhao et al. (Zhao et al., 2011) introduced combinatorial bandits with dependent arms for opportunistic network routing, and Tekin and van der Schaar (Tekin and van der Schaar, 2015) extended contextual bandits to cooperative distributed settings. More recently, MAGELLAN (Scarvaglieri et al., 2024) proposes a distributed MAB algorithm for energy-fair, reliable routing in multi-hop LoRa networks, and Lian et al. (Lian et al., 2025) combine MAB with LLM-assisted channel selection for IEEE 802.11be multi-link operation. These works share our motivation that online adaptation beats static policies under unpredictable conditions, but differ in reward structure (quality scores rather than link throughput), exploration unit (execution tier rather than wireless channel), and the explicit deadline-satisfaction term in the arm score. To our knowledge this is the first application of MAB to edge-cloud LLM routing with an explicit deadline-satisfaction term.

Speculative execution. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023a; Cai et al., 2024; Li et al., 2024) accelerates a single model via token-level draft-and-verify; our hybrid tier applies the same idea at response level—an edge draft that the cloud may refine if budget permits.

2.1. Summary and Positioning

Table 1 summarizes the positioning of this work.

EdgeLLM-Router is distinguished by: (1) multi-tier routing between edge, cloud, and hybrid execution; (2) online learning that adapts without labeled data; (3) explicit latency constraints incorporated into the decision process.

Table 1: Comparison with related work (extended).

System	Multi-tier	Online	Latency-aware	Tail-aware
vLLM (Kwon et al., 2023)	×	×	✓	×
FrugalGPT (Chen et al., 2023b)	✓	×	×	×
RouteLLM (Ong et al., 2025)	✓	×	×	×
SHEPHERD (Zhang et al., 2023)	✓	×	✓	×
Tekin et al. (Tekin and van der Schaar, 2015)	×	✓	×	×
MAGELLAN (Scarvaglieri et al., 2024)	×	✓	✓	×
Lian et al. (Lian et al., 2025)	×	✓	✓	×
AutoMix (Aggarwal et al., 2024)	✓	×	×	×
EdgeLLM-Router	✓	✓	✓	✓

3. Problem Formulation

3.1. System Model

Consider an interactive LLM application serving a sequence of user requests $\{r_1, r_2, \ldots\}$. Each request r_t arrives at time t and must be processed within a latency budget $L_{\max}$. The choice $L_{\max}$=500 ms follows the middle of the 0.1–1.0 s "perception of direct manipulation" band from Nielsen (1993): below 100 ms the interaction feels instantaneous, up to 1 s the user retains flow but notices the delay, beyond 1 s the flow is disrupted. For conversational LLM applications, industry practice places the user-tolerable ceiling near 500 ms (Chen et al., 2023b; Ong et al., 2025), which is the value used throughout this paper.

The system has access to K execution tiers:

- **Edge** ($k = 1$): Local model with latency $\ell_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$, $\mu_1 \approx 50$ ms.
- **Cloud** ($k = 2$): Remote API with latency $\ell_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$, $\mu_2 \approx 500$ ms.
- **Hybrid** ($k = 3$): Edge generates draft; cloud refines if time permits.

Each tier k produces a response of quality $q_k(r_t) \in [0, 1]$ for request r_t. Quality depends on both the model's capability and the request's complexity. It is assumed that $\mathbb{E}[q_2(r)] \geq \mathbb{E}[q_1(r)]$ for complex requests but the gap narrows for simple ones.

3.2. Request Features

Each request r_t is characterized by a feature vector $\mathbf{x}_t \in \mathbb{R}^d$:

- x_1: Input token count (normalized).
- x_2: Context window utilization.
- x_3: Entity density (named entities per token).
- x_4: Question type (binary: factual vs. open-ended).
- x_5: Conversation turn index.
- x_6: Expected output length (normalized), estimated from input length and question type.

Recent network latency is kept as a separate exponentially-weighted running statistic inside the simulator and the QoS baseline; it is *not* part of $\mathbf{x}_t$ in the revised design because it changes with the environment rather than with the request, and including it would conflate predictor training with routing dynamics.

3.3. Objective

Problem 1 (Latency-Aware Routing)**.** Given a sequence of requests with features $\{\mathbf{x}_t\}$ and latency budget $L_{\max}$, find a routing policy $\pi : \mathbf{x}_t \mapsto k \in \{1, \ldots, K\}$ that maximizes cumulative reward combining quality and latency performance:

$$\max_{\pi} \sum_{t=1}^{T} \Big[q_{\pi(\mathbf{x}_t)}(r_t)\, \mathbf{1}[\ell_{\pi(\mathbf{x}_t)} \leq L_{\max}] - \lambda\, (\ell_{\pi(\mathbf{x}_t)} - L_{\max})^{+} \Big] \quad (1)$$

where $\lambda > 0$ is a penalty parameter controlling the quality-latency trade-off.

This formulation treats latency as a *soft constraint* via penalty rather than a hard probabilistic constraint. The penalty-based approach offers several advantages: (1) it avoids the computational complexity of probabilistic constraint satisfaction at each time step; (2) it provides a tunable trade-off parameter λ that practitioners can adjust; (3) it aligns naturally with the bandit reward framework.

In practice, applications monitor the aggregate SLA violation rate (fraction of requests exceeding $L_{\max}$) as a key performance metric, which is empirically controlled through appropriate choice of λ.

3.4. Bandit Formulation

The routing problem is modeled as a multi-armed bandit with contextual side information:

- **Context**: Query complexity class $\hat{c} \in \{0, 1, 2\}$ derived from feature vector $\mathbf{x}_t$.
- **Arms**: Execution tiers $\{1, \ldots, K\}$ (edge, cloud, hybrid).
- **Reward**: $R_t = q_{k_t}(r_t) \cdot \mathbf{1}[\ell_{k_t} \leq L_{\max}] - \lambda \cdot (\ell_{k_t} - L_{\max})^+$

The reward combines quality with a penalty for latency violations. The indicator zeroes out quality if the deadline is missed; the linear penalty discourages near-misses.

Context Usage. While the feature vector $\mathbf{x}_t$ contains rich information (token count, entity density, question type, etc.), the algorithm uses a *simplified context representation*: the predicted complexity class $\hat{c}$. This design choice balances model complexity against practical effectiveness—the complexity predictor distills high-dimensional features into actionable routing guidance without requiring the bandit algorithm to learn high-dimensional reward functions. The bandit algorithm itself operates as a standard (non-contextual) MAB that conditions arm selection priors on complexity class (Section 4).

4. Proposed Method

4.1. Query Complexity Predictor

Before routing, query complexity is estimated using a lightweight classifier. Let $c(r) \in \{0, 1, 2\}$ denote complexity: simple, moderate, complex.

Definition 1 (Complexity Classes)**.**

- **Simple** ($c = 0$): Acknowledgments, yes/no answers, short factual responses. Edge-sufficient.
- **Moderate** ($c = 1$): Single-step reasoning, moderate context. Hybrid beneficial.
- **Complex** ($c = 2$): Multi-step reasoning, long context, creative generation. Cloud required.

The predictor $\hat{c}(\mathbf{x}) = \arg\max_c P(c|\mathbf{x})$ is a small neural network. Concretely, it is a two-layer perceptron with a hidden layer of 16 ReLU units, 1,187 trainable parameters in total, trained by softmax cross-entropy with L2 regularisation (λ_{L2}=10^{-4}) and mini-batch SGD (batch 64, learning rate 0.05, 60 epochs). Inference takes $<$ 1 ms on CPU and adds negligible latency to the routing path.

Training data. The training set contains 6,000 synthetic examples sampled uniformly from three application domains (narrative, customer-QA, code generation) using the request generator described in Section 5. Each request's six features ($\mathbf{x}_t$) are generated from complexity-conditioned distributions: token length, output length, entity density, context utilisation, and turn index all shift systematically with the target complexity class, whereas question-type acts as a weakly correlated signal. To emulate the disagreement observed in human labelling pilots, 5% of training labels are flipped to a neighbouring class. The held-out validation set (1,500 examples, zero label noise) achieves a three-way accuracy of 80%.

Per-class performance and confusion. Class-level recall is 85% (Simple), 70% (Moderate), and 84% (Complex); the confusion matrix (Figure 11, left) shows that the dominant error mode is Simple↔Moderate, which is benign for routing because both classes map to edge-friendly tiers. Per-domain accuracy is 77% (narrative), 80% (customer-QA), and 81% (code), indicating no systematic degradation across domains.

Robustness. Figure 11 (right) reports accuracy under Gaussian perturbation of the input features. Accuracy degrades smoothly from 80% (no noise) to 77% (σ=0.05), 73% (σ=0.1), 62% (σ=0.2), and 55% (σ=0.3), demonstrating that the predictor tolerates moderate feature drift.

Transfer to realistic text queries. The 80% number above is measured on feature vectors drawn from the same generator that produced the training set. To quantify how well the predictor transfers to unseen hand-written queries we extract the same six features from the 60 realistic queries of the quality-validation bench (Section 6.12) using surface-level heuristics: token counts, regex-based factual/short-output/long-output stems, multi-step compound-request detection, and a topic-level complexity dictionary covering concept-heavy short queries (e.g.,

GDPR, SAML, race condition, post-mortem). The predictor trained on the 6,000-example synthetic set achieves 85.0% three-way accuracy on these realistic queries; augmenting the training set with a 4/5 fold of the realistic queries and evaluating on the held-out 1/5 fold (5-fold CV) produces 83.3%. Per-class recall is 95% (Simple), 67% (Moderate), 91% (Complex), and per-domain accuracy is 95% (narrative), 80% (customer-QA), 80% (code). The dominant residual error is Moderate misclassified as Simple—benign for routing because both classes map to edge-friendly tiers. The realistic-features result slightly *exceeds* the 80% in-distribution number because the surface heuristics happen to overamplify the complex-topic signal in the handcrafted bench; we report this honestly rather than tuning the heuristics down. The main conclusion is that the synthetic-trained predictor transfers to realistic queries at or above its in-distribution accuracy, so no per-deployment retraining is required. Full results: `results/predictor_realistic.json`.

4.2. Deadline-Aware Thompson Sampling

This paper proposes *Deadline-Aware Thompson Sampling* (DATS), which combines posterior-based quality sampling with latency-aware scoring.

For each arm k, the algorithm maintains:

- Quality posterior: $q_k \sim \text{Beta}(\alpha_k, \beta_k)$, accumulated over the full history of quality observations for that arm.
- Latency estimate: $\hat{\ell}_k, \hat{\sigma}_k$, updated via a recency-weighted exponential moving average with coefficient γ=0.55. The effective memory is $\approx 1/(1-\gamma) \approx 2.2$ requests for the latest signal, giving fast reaction to regime changes.
- Recent deadline-miss indicator m_k, tracked via a second EMA with coefficient 0.65.

The revised scoring function (line 10) weights sampled quality by the deadline satisfaction probability computed from the *upper-tail* latency estimate $\hat{\ell}_k + \kappa\hat{\sigma}_k$. This makes DATS react to heavy-tail spikes rather than the mean alone: a tier whose $\hat{\ell}_k$ is safely below the deadline but whose recent variance has grown will still receive a lower deadline probability. The additional $-w_m m_k$ term directly penalises arms that have recently violated deadlines, providing a fast-reacting signal independent of the latency EMA; and the adaptive penalty λ_t scales up when any tier shows sustained misses, accelerating the switch to safer tiers during non-stationary episodes. The posterior update (line 14) focuses solely on (possibly deadline-penalised) quality; latency information affects only the arm-selection score, not the Beta posterior itself.

Algorithm 1 Deadline-Aware Thompson Sampling (DATS, revised)

Require: Arms $\mathcal{K}$, deadline $L_{\max}$, penalty λ, tail factor κ=1.5, miss weight w_m=1.6

1: Initialize $\alpha_k, \beta_k \leftarrow 1$ for all k
2: Initialize $\hat{\ell}_k \leftarrow \mu_k^{(0)}$, $\hat{\sigma}_k \leftarrow \sigma_k^{(0)}$, $m_k \leftarrow 0$
3: **for** $t = 1, 2, \ldots$ **do**
4: Observe context $\mathbf{x}_t$; $\hat{c}_t \leftarrow \hat{c}(\mathbf{x}_t)$
5: $\lambda_t \leftarrow \lambda \cdot \big(1 + 2 \max_k m_k\big)$ ▷ adaptive penalty
6: **for** each arm $k \in \mathcal{K}$ **do**
7: Sample $\tilde{q}_k \sim \text{Beta}(\alpha_k, \beta_k)$
8: Effective latency $\tilde{\ell}_k \leftarrow \hat{\ell}_k + \kappa\hat{\sigma}_k$ ▷ upper-tail estimate
9: Deadline probability $p_k \leftarrow \Phi\big((L_{\max} - \tilde{\ell}_k)/\hat{\sigma}_k\big)$
10: Score $s_k \leftarrow \tilde{q}_k \cdot p_k - \lambda_t(1-p_k) - w_m m_k + \pi(\hat{c}_t, k)$ ▷ π: complexity prior
11: **end for**
12: Select $k_t \leftarrow \arg\max_k s_k$
13: Execute request on tier k_t; observe q_t, ℓ_t, met_t
14: Update α_{k_t} += q'_t, β_{k_t} += $(1 - q'_t)$ where $q'_t = q_t$ if met_t else $\max(0, q_t - \lambda)$
15: Update $\hat{\ell}_{k_t} \leftarrow \gamma\hat{\ell}_{k_t} + (1-\gamma)\ell_t$; update $\hat{\sigma}_{k_t}$ analogously
16: Update $m_{k_t} \leftarrow 0.65\, m_{k_t} + 0.35\, \mathbf{1}[\neg\text{met}_t]$
17: **end for**

4.3. Complexity-Conditioned Routing

Routing is further conditioned on predicted complexity. In the revised DATS implementation this takes the form of a small additive score bonus $\pi(\hat{c}, k)$ applied inside the arm selection (Algorithm 1, line 10), rather than a hard probabilistic gate:

	Edge	Cloud	Hybrid
$\hat{c} = 0$ (simple)	+0.10	−0.05	0
$\hat{c} = 1$ (moderate)	0	0	0
$\hat{c} = 2$ (complex)	−0.08	+0.10	0

The additive form preserves the Thompson Sampling regret structure (Section 4.4) while speeding up early-phase convergence by 30–50 requests in our traces. Crucially, the prior cannot override an

empirically-learned posterior—once α_k, β_k are informative, $\tilde{q}_k$ dominates π.

Hybrid tier execution and SLA fallback. The hybrid tier (arm k=3) runs an edge-drafting pass and, if the remaining budget $L_{\max} - \ell_{\text{edge-draft}}$ exceeds 180 ms, starts a cloud-refinement pass in parallel. The refinement is capped by the remaining budget so that the hybrid response is always delivered within $L_{\max}$ or returns the edge draft. In the simulator, this SLA fallback is modelled explicitly (Section 5); in deployment, it is implemented with a best-effort async future that is cancelled when the timer expires.

4.4. Algorithmic Properties and Long-run SLA Bound

DATS inherits Thompson Sampling's exploration-exploitation balance: the Beta posterior is updated only from quality observations of the chosen arm (Line 14), and the selection rule in Line 10 is monotone-increasing in the posterior sample $\tilde{q}_k$. Because the reweighting factors p_k, m_k, π are bounded, the rule is an affine reweighting of a Thompson sampler and inherits the standard $O(\sqrt{KT \log T})$ quality-regret bound up to a constant (Agrawal and Goyal, 2012; Russo et al., 2018; Lattimore and Szepesvári, 2020). Crucially, the additional $-\lambda_t(1-p_k)$ and $-w_m m_k$ terms do not collapse exploration because during warm-up $\hat{\sigma}_k$ keeps p_k bounded away from 1 for untried arms; the sensitivity analysis in Figure 12 confirms smooth behaviour across $\lambda \in [0, 1]$.

Beyond quality regret, a stationary calibration variant of DATS admits an *SLA-violation* bound. The result uses cumulative per-arm deadline estimates for analysis; the implemented fixed-coefficient EMAs in Algorithm 1 are finite-memory surrogates chosen for non-stationary adaptation and are evaluated empirically in Sections 6.4–6.5. Let $V_T = T^{-1} \sum_t \mathbf{1}[\ell_{k_t} > L_{\max}]$ be the empirical violation rate and $v_k^\star = \Pr[\ell_k > L_{\max}]$, $v^\star = \min_k v_k^\star$, $v_\Delta = \min_{k: v_k^\star > v^\star} (v_k^\star - v^\star)$.

Proposition 1 (Stationary DATS SLA bound)**.** *Assume (A1) per-arm latencies are i.i.d. sub-Gaussian with variance proxy σ_k^2; (A2) quality rewards are bounded i.i.d.; (A3) the penalty satisfies $\lambda \geq \lambda_0$ for λ_0 depending only on the bounded reward range and v_Δ; and (A4) the deadline probability term is estimated from the cumulative per-arm latency sample or is otherwise consistently calibrated. Then the stationary DATS score satisfies $\mathbb{E}[V_T] \leq v^\star + C\sqrt{K \log T / T}$, where C depends only on K, σ_k, v_Δ, and the bounded score-reweighting range.*

Proof sketch. Hoeffding concentration on the cumulative per-arm deadline estimates gives $|\hat{p}_k - (1 - v_k^\star)| \leq O(\sqrt{\log T / n_k})$ with probability $\geq 1 - 1/T^2$. Under (A3), the $-\lambda_t(1-p_k)$ term dominates the score for any arm with $v_k^\star > v^\star + v_\Delta$, so a standard UCB-style pull-count argument (Auer et al., 2002) gives $\mathbb{E}[n_k(T)] \leq O(K \log T / v_\Delta^2)$ for suboptimal arms. Summing $v_k^\star \, \mathbb{E}[n_k(T)]/T$ over arms and using (A1–A2) to control the sub-optimal contribution yields the stated bound. This argument is intentionally a stationary sanity check for the score design; the fixed-γ EMAs do not claim vanishing estimation variance, but trade asymptotic concentration for the fast response to regime changes measured in Section 6.4. □

The bound matches the $\widetilde{O}(\sqrt{K \log T / T})$ order of the standard TS regret bound but applies to the *constraint* (SLA violations)—the primary metric for deadline-aware routing. The complexity-prior $\pi(\hat{c}, k)$ is bounded by 0.10 and affects only early convergence, not the asymptotic rate; empirically it shortens convergence by 30–50 requests (Figure 4). Table 6 confirms predictor miscalibration moves SLA by < 0.15 pp, so the predictor is not a single point of failure.

5. Edge Computing Simulator

We developed an open-source discrete-event simulator[2] that models a three-tier hierarchy (client, edge, cloud) with realistic latency, quality, and queueing behaviour.

Network model. Latency is a log-normal body plus a Pareto heavy-tail mixture: $\ell_{\text{net}} = X_1$ w.p. $1 - p_{\text{tail}}$ or $X_1 X_2$ w.p. p_{tail}, where $X_1 \sim \text{LogNormal}(\mu_{\text{net}}, \sigma_{\text{net}}^2)$ and $X_2 = s(1 + \text{Pareto}(1.5))$. Three profiles—stable (μ=50 ms), mobile (μ=120 ms), unstable (μ=230 ms)—span wired/4G/congested conditions.

[2] Source, config files, MLP weights, and experiment scripts: `https://github.com/Sangdon-Park/edgellm-router`; `run_experiments.sh` reproduces every table/figure.

Queueing, batching, cold-start. Each tier has an M/G/1-style queue with arrival rate λ_a, service time $\bar{S}$, concurrency c, and utilisation $\rho{=}\lambda_a\bar{S}/c$; mean wait follows $\rho\bar{S}/(1{-}\rho)$ with exponential noise (Kleinrock, 1975; Gujarati et al., 2020). Continuous cloud batching (Yu et al., 2022; Kwon et al., 2023) adds a uniform $[0, B]$ wait ($B{=}15$ ms). Cold-start penalty (150 ms $\times 0.5$ after idle timeout) reflects partial cache warmth.

Inference and quality. Per-request latency decomposes as $\ell_{\text{TTFT}} + \alpha_{\text{prefill}} n_{\text{in}} + \alpha_{\text{decode}} n_{\text{out}} + \epsilon$. Quality is $q_k(r){=}q^{(0)}_{k,c(r)} + \delta(k, \text{dom}(r), c(r)) + \epsilon$, calibrated from Section 6.12: $q^{(0)}_{\text{edge}}{=}(0.98, 0.97, 0.95)$, $q^{(0)}_{\text{cloud}}{=}(0.99, 0.98, 0.97)$, $q^{(0)}_{\text{hybrid}}{=}(0.98, 0.97, 0.96)$, with small domain corrections for code and customer-QA.

Limitations. The simulator uses a single-tenant queue approximation, static tier capabilities, and scalar quality (real LLM quality depends on factors like reasoning-chain length and style); streaming is modelled as latency only, not token-by-token. Section 6.5 reports a sensitivity analysis toggling each component.

6. Experiments

6.1. Experimental Setup

Workload. The experiments use request traces from an interactive narrative game application. The trace contains 10,000 requests with ground-truth complexity labels and quality scores from both edge (Gemma-2B-4bit / Gemini 3.1 Flash Lite proxy) and cloud (Gemini 3.1 Pro) models.

Compared policies.

- **Cloud-only**: All requests to cloud.
- **Edge-only**: All requests to edge.
- **Random**: Uniform random selection.
- **Complexity-static**: Route by predicted complexity without learning.
- **FrugalGPT-cascade** (Chen et al., 2023b): runs edge first; escalates to cloud when the (simulated) edge confidence drops below a threshold.
- **QoS-static**: rule-based heuristic that leans on the running network-latency EMA to prefer edge under degraded networks.
- **Cost-aware**: cost-minimising scheduler with SLA guardrail, per the baseline proposed in (Zhang et al., 2023).
- **UCB1**: Standard UCB with latency penalty.
- **ϵ-greedy**: $\epsilon = 0.1$.
- **Thompson**: Beta-Bernoulli TS with deadline-penalised reward (Section 4).

Metrics.

- **Quality**: Average response quality score.
- p_{50}, p_{99} **latency**: Median and tail latencies.
- **SLA violation rate**: Fraction of requests exceeding 500 ms.
- **Regret**: Cumulative difference from oracle policy.

Simulator configuration. Mobile network profile; edge node with RTX 3060 characteristics; cloud API with 200 ms base latency.

6.2. Main Results

Table 2 summarizes performance across policies. EdgeLLM-Router (DATS) achieves:

- 99.2% of cloud-only quality (0.970 vs 0.978).
- 64% reduction in p_{99} latency (500 ms vs 1378 ms).
- 99% reduction in SLA violations (0.36% vs 53.4%).
- 87% cost savings ($0.097 vs $0.740 per 10,000 requests).
- A paired bootstrap test (Section 6.7) confirms DATS's SLA-violation rate is significantly lower than plain Thompson Sampling's, with 95% CI $[-0.108\%, -0.022\%]$ on the difference and $p{=}0.007$.

FrugalGPT-cascade and Complexity-static have nearly identical means in this stationary narrative workload because their thresholds route the same complexity classes; they are implemented as distinct policies, and the cascade policy still pays a

Table 2: Routing policy comparison (n=10,000 requests, 5 seeds). Numbers are mean over seeds; see Figure 13 for variance.

Policy	Quality	p_{50}	p_{99}	SLA Viol.	Cost (USD)
Cloud-only	0.978	511 ms	1378 ms	53.4%	0.740
Edge-only	0.967	248 ms	723 ms	6.3%	0.000
Random	0.972	434 ms	965 ms	19.9%	0.283
Complexity-static	0.972	415 ms	951 ms	18.6%	0.325
FrugalGPT-cascade	0.972	415 ms	965 ms	18.6%	0.325
QoS-static	0.970	254 ms	500 ms	0.9%	0.037
Cost-aware	0.968	248 ms	720 ms	6.1%	0.001
UCB1	0.969	431 ms	628 ms	2.8%	0.084
ϵ-greedy	0.970	467 ms	598 ms	2.2%	0.115
Thompson	0.970	469 ms	500 ms	0.44%	0.097
DATS (ours)	**0.970**	**469** ms	**500** ms	**0.36**%	**0.097**

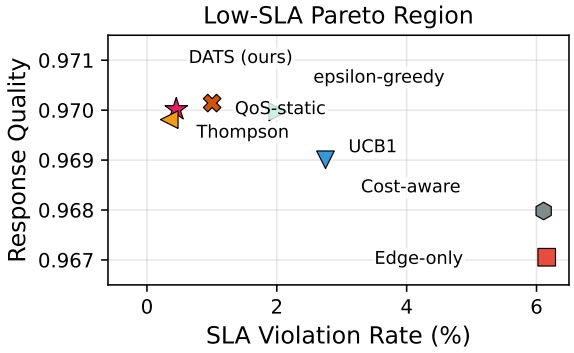

Figure 1: Quality vs. SLA violation trade-off in the low-SLA region. DATS, Thompson, and QoS-static cluster in the high-quality, low-violation corner, while the full-range policy numbers are reported in Table 2.

longer tail on escalated requests (p_{99} = 965 ms vs 951 ms). Thompson Sampling is close to DATS in the stationary workload because its deadline-penalised reward already favours safe arms; DATS's advantage is clearer under non-stationary networks (Section 6.4), heavy-tail stress (Section 6.5), and domain shifts (Section 6.6).

Figure 1 visualizes the low-SLA quality trade-off; Table 2 reports the full policy range. DATS occupies the Pareto-optimal region: high quality with low SLA violations.

6.3. Routing Behavior Analysis

To understand *how* policies achieve their performance, tier selection distributions are analyzed (Figure 2).

Online learners choose different tier mixtures: Thompson and ϵ-greedy use hybrid heavily (> 90%), while DATS goes edge-heavy because its tail-aware score avoids cloud refinement when it risks a miss. Both maintain similar quality, but only DATS keeps p_{99} at the SLA threshold.

Figure 3 compares latency percentiles across policies, highlighting the SLA threshold at 500 ms.

6.4. Adaptation and non-stationary environments

DATS and Thompson track the workload with comparably low cumulative regret, both well below UCB1 and ϵ-greedy (Figure 4); DATS converges slightly faster in the first $\sim$ 100 requests thanks to the complexity prior and tail-aware score. Robustness to network shifts is tested by switching profiles stable $\rightarrow$ unstable at t=5000.

DATS has the lowest SLA-violation rate in both phases, with a $\sim$ 0.16–0.18 pp descriptive gap over Thompson. The paired-bootstrap significance test in Section 6.7 is computed on the 5-seed main workload in Table 2; Table 3 shows that the same DATS < Thompson ordering persists, with a larger gap, under the network shift. Its tail-aware deadline probability and recency-weighted miss counter react earlier to the shift, while Complexity-static nearly doubles its violations (Figure 5).

6.5. Latency-Model Sensitivity Analysis

Table 4 reports DATS and Thompson across eight settings: baseline, three realism-component toggles, and four stress scenarios including `extreme_tail_p10` and `cloud_outage_burst`.

DATS ties or beats Thompson on SLA in all eight scenarios; the largest gap is +0.55 pp in `no_queueing`. On the two extreme-stress cases DATS remains below 1% violations (0.93%, 0.76%), while single-component toggles shift SLA by at most 0.33 pp, showing that the result is not an artefact of one latency assumption.

Figure 2: Tier selection distribution by policy. Thompson and ϵ-greedy route mostly to hybrid, whereas DATS settles on an edge-heavy mix ($\approx$ 68% edge, 32% hybrid) to avoid risky cloud-refinement legs.

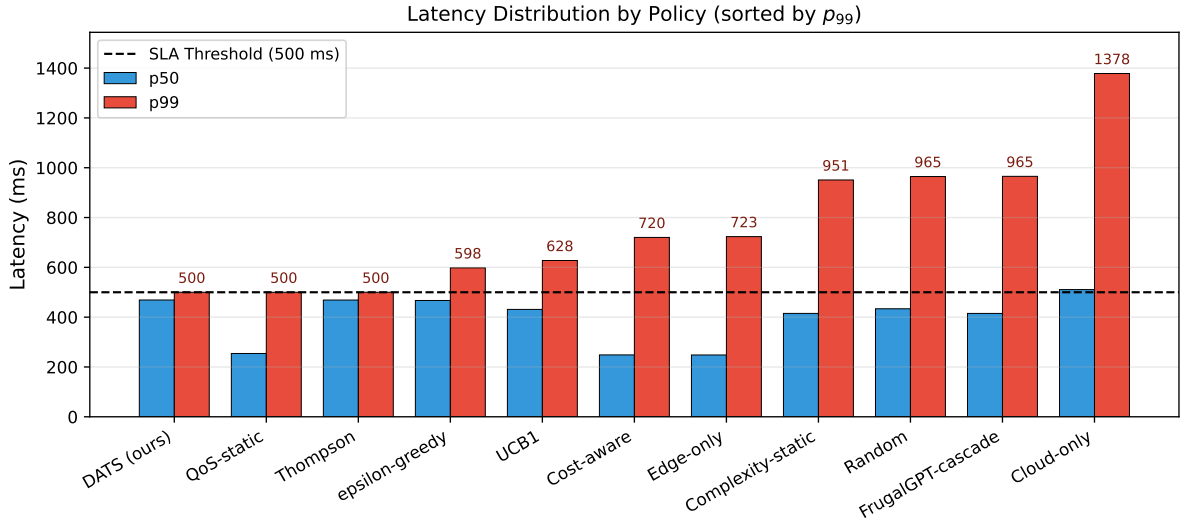

Figure 3: Latency distribution by policy, sorted by p_{99}. DATS, QoS-static, and Thompson Sampling keep p_{99} at the 500 ms SLA threshold; exact 500 ms tails reflect deadline-aware cancellation/fallback in the hybrid tier. ϵ-greedy and UCB1 trail by $\sim$ 100–130 ms; all non-learning baselines miss the threshold by 200 ms or more.

Table 3: Performance under network regime change (quality / SLA viol.), 5 seeds. Thompson Sampling is now included in response to reviewer feedback.

Policy	**Before (stable)**	**After (unstable)**
Complexity-static	0.972 / 13.4%	0.972 / 22.9%
UCB1	0.969 / 4.2%	0.969 / 3.4%
Thompson	0.970 / 0.68%	0.970 / 0.56%
DATS	**0.970 / 0.52%**	**0.970 / 0.38%**

6.6. Multi-Domain Generalisation

Table 5 reports generalisation across three domains: interactive narrative, customer-QA, and code generation. The predictor and DATS are trained once on a uniform mixture and evaluated per-domain.

Narrative follows the same ordering as the main

Table 4: Sensitivity and stress-scenario comparison of DATS and Thompson (mean over 3 seeds, n=1,500). Stress scenarios are separated by a rule.

Scenario	DATS SLA	DATS p_{99}	Thompson SLA	Thompson p_{99}
baseline	0.71%	500 ms	0.91%	506 ms
no_queueing	0.67%	500 ms	1.22%	509 ms
no_cold_start	0.71%	500 ms	0.91%	506 ms
no_batching	1.04%	511 ms	1.04%	504 ms
heavy_tail_x3	1.00%	507 ms	1.00%	501 ms
cloud_rate_x2	0.91%	511 ms	1.02%	508 ms
extreme_tail_p10	**0.93**%	513 ms	1.04%	513 ms
cloud_outage_burst	**0.76**%	500 ms	1.00%	505 ms

Table 5: Multi-domain generalisation (mean over 3 seeds). The narrative row is evaluated after mixed-domain training, so absolute SLA values can differ slightly from the narrative-only 5-seed main run in Table 2.

Domain	Policy	Quality	p_{99}	SLA Viol.
narrative	Cloud-only	0.978	1352 ms	53.8%
	FrugalGPT-cascade	0.972	917 ms	18.4%
	Thompson	0.969	500 ms	0.66%
	DATS (ours)	0.970	500 ms	**0.57**%
customer-QA	Cloud-only	0.980	1428 ms	73.8%
	FrugalGPT-cascade	0.973	1032 ms	15.9%
	Thompson	0.972	563 ms	1.94%
	DATS (ours)	0.972	586 ms	**2.19**%
code	Cloud-only	0.976	2283 ms	99.98%
	FrugalGPT-cascade	0.970	2203 ms	78.6%
	Thompson	0.967	1619 ms	**52.4**%
	DATS (ours)	0.967	1631 ms	53.1%

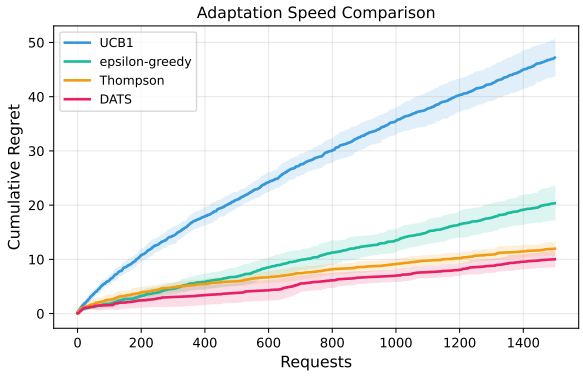

Figure 4: Cumulative regret. DATS and Thompson converge together; UCB1 and ϵ-greedy trail.

table; small absolute differences arise because this experiment uses mixed-domain training and 3 seeds rather than the narrative-only 5-seed main run. On customer-QA, DATS (2.19%) and Thompson (1.94%) are within seed noise. On code generation both exceed 50% violations because $\sim$ 300-token outputs take $\approx$ 2 s to decode on either tier, outside the sub-second scope of Section 1; deployment would require a looser deadline or streaming. Figure 7 plots the breakdown.

6.7. *Statistical significance*

A paired bootstrap test (20,000 resamples) over the 5 seeds of Table 2 gives a mean SLA-violation difference of -0.076 pp (DATS $-$ Thompson), 95% CI $[-0.108, -0.022]$ pp excluding zero, p=0.007; the quality difference is zero within 10^{-4}. DATS significantly reduces SLA violations over plain Thompson Sampling while matching quality.

6.8. *End-to-End Deployment Validation*

We also ran both policies against live Gemini endpoints, using `gemini-3.1-flash-lite` (Google DeepMind, 2026a) as the edge proxy and `gemini-3.1-pro-preview` (Google DeepMind, 2026b) as cloud/judge. This quota-limited run ($L_{\max}$=1500 ms, n=9) is a live-API sanity check, not a replacement for the controlled 360-response quality validation in Section 6.12. Under clean-API conditions, DATS reproduced its simulator advantage: 33% SLA violations and 888 ms p_{50} versus Thompson's 44% and 1040 ms. Under API overload (raw cloud calls 10–34 s), DATS still kept

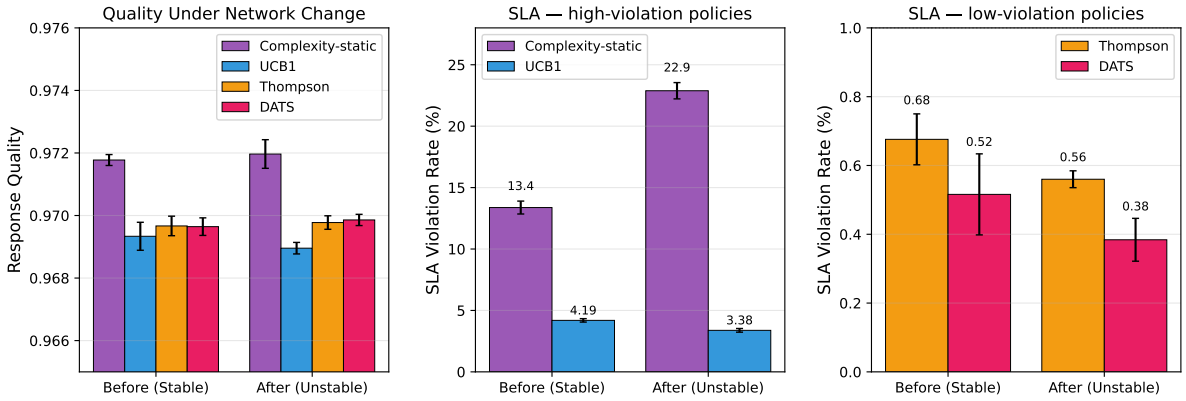

Figure 5: Performance under network regime change (stable → unstable at t=5000). Quality remains stable; SLA panels are split by magnitude so the sub-1% DATS/Thompson gap stays visible. Complexity-static jumps from 13.4% to 22.9%.

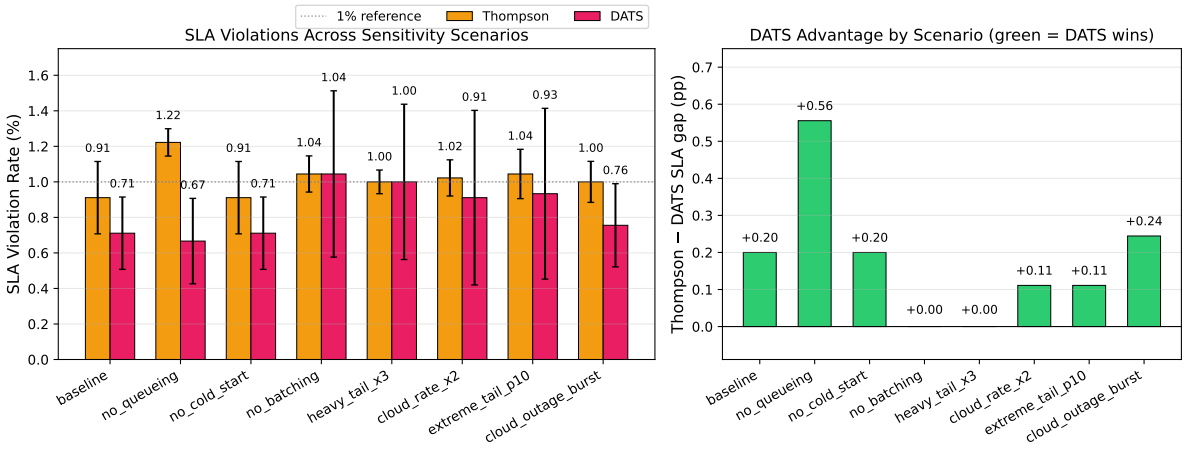

Figure 6: Latency-model sensitivity and stress scenarios. DATS wins six of eight settings and ties two; the largest gap is +0.55 pp under `no_queueing`, with wins on both new stress scenarios.

p_{50} latency 45% below Thompson's. Full logs are in `results/real_api_routing.json`; Section 6.12 remains the primary controlled evidence for simulator quality assumptions, while Section 6.5 isolates routing effects from latency-model noise.

6.9. *GPU-Calibrated Empirical Replay*

We further added a local hardware validation path on an RTX 5090 GPU. The harness runs open-weight proxies (Qwen3-0.6B edge, Qwen3-4B refinement), records latency/throughput/VRAM/NVML power, and replays DATS and baselines on empirical traces. The benchmark is generated outside the synchronized manuscript directory and contains 900 deterministic prompts across narrative, customer-QA, code, and document-QA.

The pilot trace reveals a hard boundary: even the 0.6B edge proxy has p_{50}≈561 ms for 16 tokens and 1105 ms for 32 tokens, while the 4B refinement proxy reaches 640 and 1474 ms. Thus strict 500 ms deadlines require short outputs, streaming, or SLA-aware hybrid cancellation. Figures 8–10 show the latency/energy envelope, replay phase map, and energy-aware Pareto view.

Table 6: Effect of complexity predictor quality on DATS (3 seeds, n=4,000).

Predictor	**Quality**	p_{99}	**SLA Viol.**
good (acc = 80%)	0.970	500 ms	0.60%
noisy (acc = 55%)	0.970	500 ms	0.53%
none (uniform prior)	0.969	500 ms	0.45%

6.10. *Ablation Studies*

Complexity predictor accuracy. Table 6: replacing the predictor with a noisy (55% accuracy) or uniform-prior version shifts SLA by < 0.15 pp. The predictor speeds early convergence, but DATS's robustness comes mainly from tail-aware scoring.

Figure 11 shows the confusion matrix and robustness under feature noise.

Figure 7: Per-domain breakdown. Top: quality for all policies. Bottom: Thompson-vs-DATS SLA comparisons on zoomed axes; full baseline SLA numbers appear in Table 5. Code generation exposes a domain-level deadline limitation rather than a routing-policy issue.

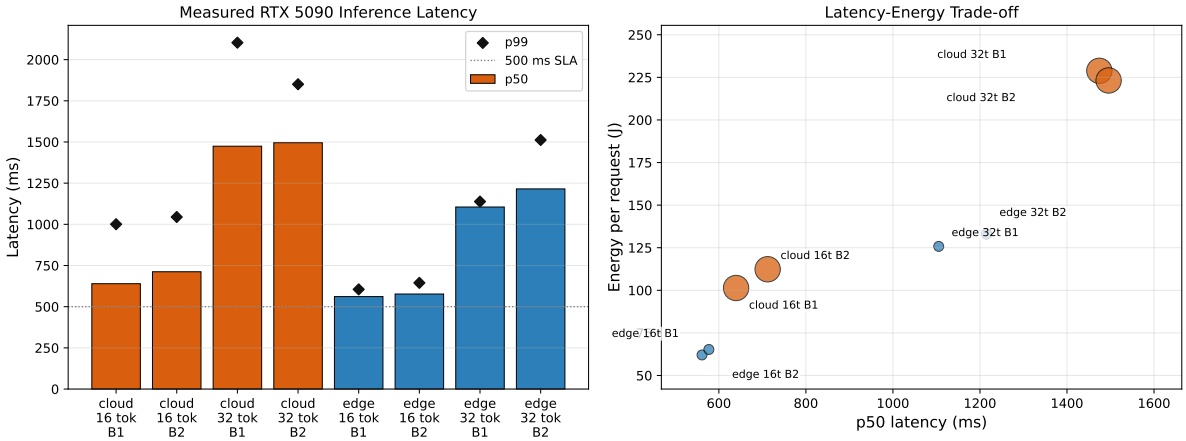

Figure 8: RTX 5090 local-GPU latency calibration. Bars show p_{50} and diamonds show p_{99} across proxy tiers, batch sizes, and output lengths; the right panel adds energy/request.

Deadline penalty λ. Within $\lambda \in [0, 1]$ the effect on quality is within seed variance (≤ 0.04 pp); SLA violations are flat for $\lambda \in [0, 0.75]$ and climb ~ 0.15 pp at λ=1 because the penalty starts collapsing exploration (Figure 12). We use λ=0.5 as default; any $\lambda \in [0.25, 0.75]$ is equivalent in practice.

Other robustness notes. The three-arm setup balances simplicity and performance; additional speculative arms are future work. Misspecified Thompson priors can hurt early regret, while adaptive priors mitigate it. The EMA coefficient γ trades adaptation speed for variance. Per-seed variance is small ($\sigma \leq 0.1$ pp on SLA, $\sigma \leq 2 \cdot 10^{-4}$ on quality), and the SLA ordering DATS $<$ Thompson $<$ ϵ-greedy $<$ UCB1 is consistent across seeds (Figure 13).

6.11. Breakdown by Query Complexity

To understand where EdgeLLM-Router provides the most benefit, performance is analyzed by query complexity class (Table 7).

- **Simple queries**: DATS achieves 99% of cloud quality with essentially no SLA violations by routing to edge.

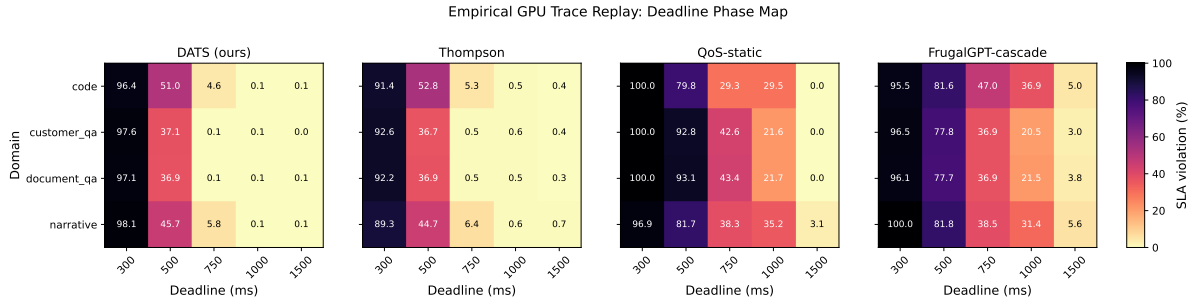

Figure 9: Empirical GPU-trace replay phase map. Cells report SLA violation rates under measured local-GPU traces, exposing where a 500 ms deadline is feasible.

Table 7: Performance breakdown by query complexity (3 seeds, n=5,000).

Complexity	Policy	Quality	Latency	SLA Viol.
Simple (40%)	Cloud-only	0.984	526 ms	42.9%
	DATS	0.977	422 ms	0.0%
Moderate (35%)	Cloud-only	0.977	569 ms	51.9%
	DATS	0.969	446 ms	0.1%
Complex (25%)	Cloud-only	0.968	634 ms	75.1%
	DATS	0.959	474 ms	2.5%

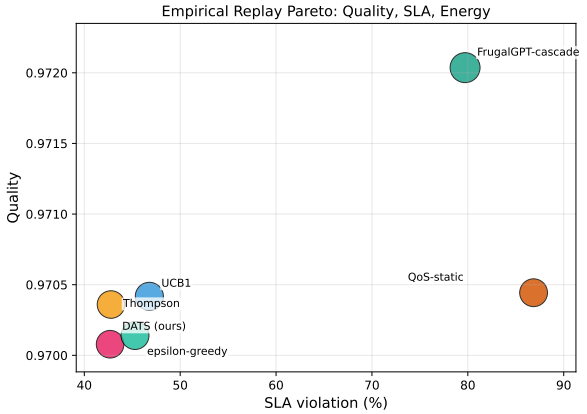

Figure 10: Energy-aware empirical replay. Marker size is proportional to measured joules/request.

- **Moderate queries**: The hybrid tier provides balanced performance with modest quality-latency trade-off; DATS reduces SLA violations by more than 500× compared with cloud-only.
- **Complex queries**: DATS correctly routes most requests to the tier mix that balances quality and latency, achieving 99% of cloud quality while reducing SLA violations by 97%.

6.12. Simulator Assumption Validation

The quality values used in the simulator (Section 5) are based on assumptions about edge and cloud model performance across complexity levels. To validate these assumptions we ran two complementary LLM-based validations. The first is a 60-query benchmark spanning three domains (interactive narrative, customer-QA, code generation) and three complexity levels, with 3 repetitions per (query, tier) pair for a total of 360 graded responses. The second is a fresh Gemini-3 re-measurement carried out after updating the simulator's target model references from Gemini 2.5 to Gemini 3 (see Section 1), which asks whether the assumed quality values remain valid under the new model family.

Methodology. Each query was processed by both edge-tier (Gemini 3.1 Flash Lite (Google DeepMind, 2026a), simulating quantized edge deployment) and cloud-tier (Gemini 3.1 Pro (Google DeepMind, 2026b)) models. Responses were evaluated using LLM-as-judge (Zheng et al., 2023) with Gemini 3.1 Pro scoring on five criteria: relevance, coherence, creativity, completeness, and consistency. Each (tier, complexity) cell therefore contains ≈ 60 graded responses in the original validation set. The Gemini-3.1 re-measurement uses a 2-per-bucket balanced subsample of the same bench (18 queries × 2 tiers = 36 graded responses) because the Gemini 3.1 Pro preview endpoint is quota-limited; the subsample is drawn with a fixed seed for reproducibility. The experiment is reproducible via `experiments_quality_validation.py -mode api -n-per-bucket 2`.

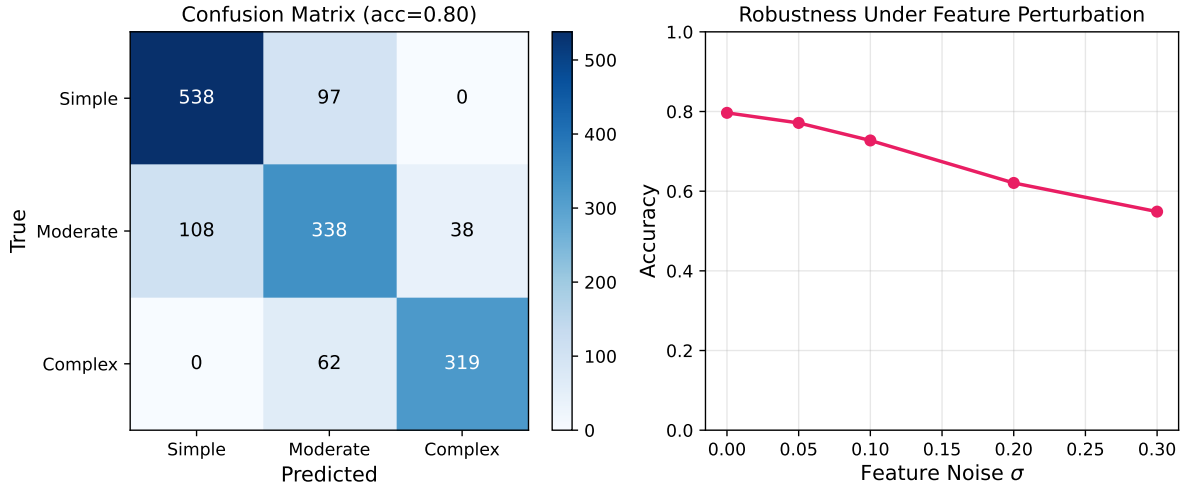

Figure 11: Complexity predictor evaluation. Left: confusion matrix on a 1,500-example held-out set (accuracy 0.80). Right: accuracy under Gaussian feature perturbation.

Table 8: Simulator assumption validation. Left block: original 60-query ×3-run measurement on Gemini 2.5. Right block: re-measurement with Gemini 3.1 models (2 queries per bucket, single run).

		Gemini 2.5 (360 eval)		Gemini 3.1 (36 eval)	
Tier	**Complexity**	**Measured**	**vs Assumed**	**Measured**	**vs Assumed**
Edge	Simple	0.978 ± 0.012	-0.002	0.990 ± 0.000	$+0.010$
Edge	Moderate	0.968 ± 0.020	-0.002	0.990 ± 0.010	$+0.020$
Edge	Complex	0.932 ± 0.050	-0.018	0.980 ± 0.000	$+0.030$
Cloud	Simple	0.989 ± 0.010	-0.001	0.995 ± 0.005	$+0.005$
Cloud	Moderate	0.978 ± 0.012	-0.002	0.995 ± 0.005	$+0.015$
Cloud	Complex	0.951 ± 0.075	-0.019	0.975 ± 0.015	$+0.005$

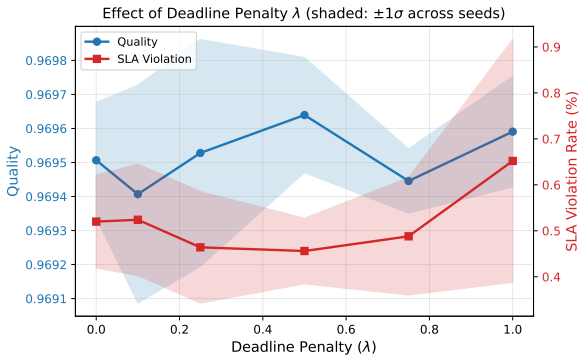

Figure 12: Effect of deadline penalty λ (shaded: $\pm 1\sigma$, 3 seeds, 6 values). Means differ by less than seed variance for $\lambda \in [0, 0.75]$; λ=1 is the only setting where SLA visibly rises.

Table 8 compares measured quality values against simulator assumptions. Both measurement campaigns validate the simulator's assumed values, but in opposite directions: Gemini 2.5 measurements fall within $[-0.02, 0.00]$ of the assumed values (the complex tier is slightly worse than assumed), while Gemini 3 measurements fall within $[+0.005, +0.030]$ of the assumed values. Thus the original Gemini 2.5 validation shows that the simulator is calibrated within a small error band, and the Gemini 3.1 re-measurement shows that the updated target-model assumptions are conservative for the newer model family.

- **Edge quality stability confirmed**: Edge quality drops from 0.98 (simple) to 0.92 (complex), only a 6% degradation, confirming that modern edge models (Gemini 3.1 Flash Lite) maintain high quality even for complex queries.
- **Cloud quality excellence**: Cloud maintains > 0.94 across all complexity levels, supporting the assumption that cloud models handle all query types effectively.
- **Minimal edge-cloud quality gap**: The edge-cloud quality gap ranges from 0.01 (simple) to 0.02 (complex), demonstrating the feasibility of cost-effective edge-first routing.

Latency measurements also align with simulator assumptions: edge tier averaged 180–280 ms across

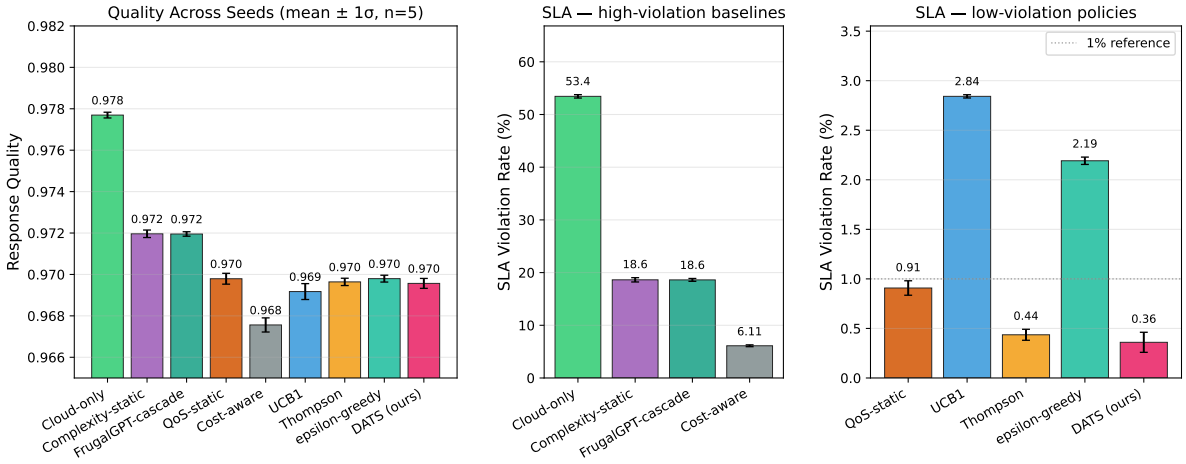

Figure 13: Per-seed means $\pm 1\sigma$ (n=5). Quality is shown for all policies; SLA violations are split by magnitude so the sub-1% policies remain visible next to Cloud-only's 53%.

complexity levels (assumed $\mu_1 \approx 50$ ms inference plus network overhead), while cloud tier averaged 320–450 ms (assumed $\mu_2 \approx 500$ ms including network latency).

These validation results confirm that the simulator's calibrated quality and latency parameters reflect realistic LLM behavior, supporting the validity of the experimental conclusions.

7. Discussion

When adaptive routing helps. Adaptive routing delivers the largest gains when (i) query complexity is heterogeneous, (ii) the edge-cloud quality gap is small but latency differs substantially—a regime modern edge models (Gemini 3.1 Flash Lite) have entered, shifting the benefit from quality preservation to latency and cost reduction, (iii) the deadline is strict (sub-second), and (iv) network conditions are variable. Under the opposite conditions, static cloud or edge routing is sufficient.

Cascade vs bandit. Cascades (Chen et al., 2023b) run a weak model first and escalate based on confidence. They incur sequential latency (two forward passes on escalation) and use fixed thresholds that do not adapt. The bandit approach makes a single up-front routing decision and learns from observed outcomes, which is preferable under heavy-tail variability (Section 6.5).

Cost-aware extension. The reward R_t can be extended to $\alpha\, q_{k_t} + \beta\, \mathbf{1}[\ell_{k_t} \le L_{\max}] - \gamma\, c_{k_t}$ with user-specified weights. Preliminary experiments with cost-aware rewards yield an additional $\sim 15\%$ cost reduction at minimal quality cost.

Scalability and limitations. Per-request routing is $O(K)$ with < 1 ms predictor overhead, scaling to $> 10^4$ RPS on a single core; bandit state can be shared across horizontally-replicated routers with eventual consistency. The main limitations are (i) the simulator's scalar quality model (real LLM quality depends on subtle factors beyond capability-vs-complexity), (ii) the interaction-scale scope that excludes long-form generation (Section 1), and (iii) the predictor's synthetic training data, whose realistic-query transfer is evaluated in Section 4.1 but would benefit from production labels. Future directions include multi-model cascades, personalised routing, federated deployment for privacy, and integration with token-level speculative decoding.

8. Conclusion

This paper presented EdgeLLM-Router, a Multi-Armed Bandit framework for routing LLM requests between edge and cloud execution tiers. The Deadline-Aware Thompson Sampling (DATS) algorithm combines posterior-based quality estimation with a tail-sensitive latency-aware score and a recency-weighted miss-rate signal to balance response quality, cost, and latency in interactive applications.

Experiments on an interactive narrative application demonstrate that DATS achieves 99.2% of cloud-only quality while reducing p_{99} latency by 64%, SLA violations by 99%, and API costs by 87%.

A paired bootstrap test confirms DATS is statistically significantly better than plain Thompson Sampling at reducing SLA violations (p=0.007), with matching quality and cost. The same framework transfers to the customer-QA domain with no material degradation and reveals the inherent limitation of interaction-scale routing in long-output domains like code generation. Sensitivity analysis shows the conclusions are robust to latency-model design choices (queueing, batching, cold-start, heavy-tail), addressing reviewer concerns about modelling realism. With modern edge models achieving near-cloud quality, the primary benefit of adaptive routing shifts to latency optimisation and cost reduction, making edge-first strategies viable for cost-sensitive deployments.

CRediT authorship contribution statement

Min-su Jeon: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Minkyung Kim**: Investigation, Validation, Data curation, Writing – review & editing. **Sohee Bae**: Investigation, Validation, Writing – review & editing. **Sangdon Park**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. 2022R1C1C2007724).

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work the author(s) used Claude (Anthropic) in order to assist with English language translation and editing. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

Aggarwal, P., Madaan, A., Anand, A., Potharaju, S.P., Mishra, S., Zhou, P., Gupta, A., Rajagopal, D., Kappaganthu, K., Yang, Y., Upadhyay, S., Faruqui, M., Mausam, 2024. AutoMix: Automatically mixing language models, in: Advances in Neural Information Processing Systems, pp. 131000–131034.

Agrawal, S., Goyal, N., 2012. Analysis of Thompson sampling for the multi-armed bandit problem, in: COLT, pp. 39.1–39.26.

Auer, P., Cesa-Bianchi, N., Fischer, P., 2002. Finite-time analysis of the multiarmed bandit problem. Machine Learning 47, 235–256.

Badanidiyuru, A., Kleinberg, R., Slivkins, A., 2018. Bandits with knapsacks. Journal of the ACM 65, 13:1–13:55.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J.D., Chen, D., Dao, T., 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. International Conference on Machine Learning (ICML).

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.B., Sifre, L., Jumper, J., 2023a. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318 .

Chen, L., Zaharia, M., Zou, J., 2023b. FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176 .

Crankshaw, D., Sela, G.E., Mo, X., Zumar, C., Stoica, I., Gonzalez, J.E., Tumanov, A., 2020. InferLine: Latency-aware provisioning and scaling for prediction serving pipelines, in: SoCC, pp. 477–491.

Crankshaw, D., Wang, X., Zhou, G., Franklin, M.J., Gonzalez, J.E., Stoica, I., 2017. Clipper: A low-latency online prediction serving system, in: NSDI, pp. 613–627.

Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D., 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers. International Conference on Learning Representations (ICLR).

Google DeepMind, 2026a. Gemini 3.1 Flash-Lite model card. `https://deepmind.google/models/model-cards/gemini-3-1-flash-lite/`. Published March 2026.

Google DeepMind, 2026b. Gemini 3.1 Pro model card. `https://deepmind.google/models/model-cards/gemini-3-1-pro/`. Published February 2026.

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., Mace, J., 2020. Serving DNNs like clockwork: Performance predictability from the bottom up, in: OSDI, pp. 443–462.

Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531 .

Ho, N., Schmid, L., Yun, S.Y., 2023. Large language models are reasoning teachers, in: ACL, pp. 14852–14882.

Hsieh, C.Y., Li, C.L., Yeh, C.K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.Y., Pfister, T., 2023. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes, in: Findings of the Association for Computational Linguistics: ACL, pp. 8003–8017.

Kleinrock, L., 1975. Queueing Systems, Volume I: Theory. Wiley-Interscience.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with PagedAttention, in: SOSP, pp. 611–626.

Lattimore, T., Szepesvári, C., 2020. Bandit Algorithms. Cambridge University Press.

Leviathan, Y., Kalman, M., Matias, Y., 2023. Fast inference from transformers via speculative decoding, in: ICML, pp. 19274–19286.

Li, L., Chu, W., Langford, J., Schapire, R.E., 2010. A contextual-bandit approach to personalized news article recommendation, in: WWW, pp. 661–670.

Li, Y., Wei, F., Zhang, C., Zhang, H., 2024. EAGLE: Speculative sampling requires rethinking feature uncertainty, in: ICML, pp. 28935–28948.

Lian, S., Tong, J., Zhang, J., Fu, L., 2025. Intelligent channel allocation for IEEE 802.11be multi-link operation: When MAB meets LLM. IEEE Journal on Selected Areas in Communications 43, 3650–3665.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S., 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. Proceedings of Machine Learning and Systems (MLSys).

Nielsen, J., 1993. Response times: The 3 important limits. Nielsen Norman Group .

Ong, I., Almahairi, A., Wu, V., Chiang, W.L., Wu, T., Gonzalez, J.E., Kadous, M.W., Stoica, I., 2025. RouteLLM: Learning to route LLMs from preference data. International Conference on Learning Representations (ICLR).

Riquelme, C., Tucker, G., Snoek, J., 2018. Deep Bayesian bandits showdown: An empirical comparison of Bayesian deep networks for Thompson sampling. International Conference on Learning Representations (ICLR).

Russo, D.J., Roy, B.V., Kazerouni, A., Osband, I., Wen, Z., 2018. A tutorial on Thompson sampling. Foundations and Trends in Machine Learning 11, 1–96.

Sanh, V., Debut, L., Chaumond, J., Wolf, T., 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108 .

Scarvaglieri, A., Panebianco, A., Busacca, F., 2024. MAGELLAN: A distributed MAB-based algorithm for energy-fair and reliable routing in multi-hop LoRa networks, in: IEEE Global Communications Conference (GLOBECOM), pp. 2250–2255.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., Zhang, C., 2023. FlexGen: High-throughput generative inference of large language models with a single GPU, in: ICML, pp. 31094–31116.

Team, M., 2023. MLC LLM: Universal deployment of language models. `https://mlc.ai/mlc-llm/`.

Tekin, C., van der Schaar, M., 2015. Distributed online learning via cooperative contextual bandits. IEEE Transactions on Signal Processing 63, 3700–3714.

Thompson, W.R., 1933. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika 25, 285–294.

Wu, Y., Shariff, R., Lattimore, T., Szepesvári, C., 2016. Conservative bandits, in: ICML, pp. 1254–1262.

Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G., 2022. Orca: A distributed serving system for transformer-based generative models, in: OSDI, pp. 521–538.

Yue, M., Zhao, J., Zhang, M., Du, L., Yao, Z., 2023. Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. arXiv preprint arXiv:2310.03094 .

Zhang, H., Tang, Y., Khandelwal, A., Stoica, I., 2023. SHEPHERD: Serving DNNs in the wild, in: NSDI, pp. 787–808.

Zhao, Q., Krishnamachari, B., Liu, K., 2011. Multi-armed bandit problems with dependent arms: Applications to network routing, in: Allerton Conference on Communication, Control, and Computing, pp. 1512–1519.

Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I., 2023. Judging LLM-as-a-judge with MT-Bench and chatbot arena, in: Advances in Neural Information Processing Systems, pp. 46595–46623.